Programming HP Robots Otto with C/C++

Developer Guide for Arduino IDE with ESP32-WROOM

HP Robots Otto was originally designed for MicroPython with an interactive REPL. This makes learning and iteration easier and faster: you can connect, type code, and see results immediately without full recompiles.

This guide shows an alternative path using the Arduino IDE and C/C++. It is compatible with the hardware, **but it's not recommended for complete beginners:** flashing/compiling takes longer than REPL-based edits, and quick iteration is affected.

You can switch between MicroPython and Arduino at any time, but you must re-flash the board when changing languages. To return to MicroPython, flash a MicroPython firmware build for ESP32-WROOM; to use Arduino again, upload an Arduino sketch.

1. Requirements

- Arduino IDE 1.8+ or Arduino IDE 2.x
- Install ESP32 board support (Preferences → Additional Boards Manager URLs):

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

- Then open: Tools → Board → select: ESP32 Dev Module (ESP32-WROOM-32)

2. Otto Reference Pin Map & Connector Guide

Built-in LED: GPIO 2

Buzzer: GPIO 25

Battery voltage sense (ADC): GPIO 39

Connector #1: GPIO 18 & 19 (Ultrasonic or SPI OLED)

Connector #2: GPIO 16 & 17 (MP3 or digital encoder)

Connector #3 (I2C): GPIO 22=SCL, 21=SDA (OLED, matrix, accelerometer)

Connector #4: GPIO 26 (tilt or button)

Connector #5: GPIO 4 (13-LED NeoPixel ring or temperature sensor)

Connector #6 (ADC): GPIO 32 (line sensor left / microphone)

Connector #7 (ADC): GPIO 33 (line sensor right / potentiometer / LDR)

Connector #8: GPIO 27 (servo angle right)

Connector #9: GPIO 15 (servo angle left)

Connector #10: GPIO 14 (servo wheel left)

Connector #11: GPIO 13 (servo wheel right)

```
+--------------------------- ESP32-WROOM Core -------------------------
---+
|  LED(2)  BUZ(25)  VBAT(39)
|
|
|
|  C1: 18,19    C2: 16,17     C3: SCL(22) SDA(21)    C4: 26
|
|  C5: 4        C6: 32(ADC)   C7: 33(ADC)
|
|  C8: 27   C9: 15   C10: 14 (Wheel L)   C11: 13 (Wheel R)
|
+-----------------------------------------------------------------------
--+
```

Connector Use (Examples)

- #1 (GPIO 18/19): Ultrasonic HC-SR04 (Trig/Echo) or SPI OLED
- #2 (GPIO 16/17): DFPlayer Mini (MP3) or generic digital encoder (A channel)
- #3 (I2C 22/21): OLED SSD1306, LED matrix (HT16K33), accelerometer (MPU6050)
- #4 (GPIO 26): Tilt switch or push button
- #5 (GPIO 4): 13-LED NeoPixel ring (WS2812/2812B)
- #6 (GPIO 32 ADC): Line sensor L / microphone
- #7 (GPIO 33 ADC): Line sensor R / potentiometer / LDR
- #8-#11: Servos (leg angles, or continuous rotation wheels)

3. Basic Code Examples

3.0 Blink the built-in LED (GPIO 2)

```
void setup() {
```

```
  pinMode(2, OUTPUT);
}


void loop() {
  digitalWrite(2, HIGH);
  delay(500);
  digitalWrite(2, LOW);
  delay(500);

}
```

3.1 Connector #5 – 13× NeoPixel Ring (GPIO 4)

```
#include <Adafruit_NeoPixel.h>
#define PIN 4
#define NUM_LEDS 13
Adafruit_NeoPixel ring(NUM_LEDS, PIN, NEO_GRB + NEO_KHZ800);

void setup(){
  ring.begin();
  ring.clear();
  ring.setBrightness(32);
  ring.show();
}

void chase(uint32_t c, uint16_t wait){
  for (int i=0;i<NUM_LEDS;i++){
    ring.clear();
    ring.setPixelColor(i, c);
    ring.show();
    delay(wait);
  }
}

void loop(){
  chase(ring.Color(255,0,0), 60);
  chase(ring.Color(0,255,0), 60);
  chase(ring.Color(0,0,255), 60);
```

```
}
```

3.2 OLED I2C (Connector #3 – GPIO 21 SDA, 22 SCL)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void setup(){
  Wire.begin(21, 22);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0,0);
  display.println("Otto!");
  display.display();
}

void loop(){}
```

3.3 Digital Encoder (Connector #2 – GPIO 16 or 17)

```
const int ENC_PIN = 16; // or 17
volatile unsigned long ticks = 0;

void IRAM_ATTR onEnc(){ ticks++; }

void setup(){
  Serial.begin(115200);
  pinMode(ENC_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENC_PIN), onEnc, FALLING);
}

void loop(){
  static unsigned long last=0;
  if (millis()-last>500){
    last=millis();
    Serial.printf("Ticks: %lu\n", ticks);
  }
}
```

3.4 Buzzer (GPIO 25)

```
#define BUZZER 25

void setup(){}

void loop(){
  tone(BUZZER, 1000); delay(200); // 1 kHz
  tone(BUZZER, 1500); delay(200);
  noTone(BUZZER); delay(200);
}
```

3.5 Battery Voltage (GPIO 39 – ADC)

```
#define VBAT 39

void setup(){ Serial.begin(115200); }

void loop(){
  int raw = analogRead(VBAT);
  float voltage = (raw / 4095.0) * 2.0 * 3.3; // adjust divider factor
  Serial.println(voltage);
  delay(500);
}
```

3.6 Servos (GPIO 27, 15, 14, 13)

```
#include <ESP32Servo.h>

Servo sRightAngle;  // 27
Servo sLeftAngle;   // 15

void setup(){
  sRightAngle.attach(27);
  sLeftAngle.attach(15);
}

void loop(){
  sRightAngle.write(90);
  sLeftAngle.write(90);
  delay(1000);
  sRightAngle.write(0);
  sLeftAngle.write(180);
```

```
  delay(1000);
}
```

3.7 Read analog sensors (GPIO 32 & 33)

```
void setup() { Serial.begin(115200); }

void loop() {

  int left = analogRead(32);

  int right = analogRead(33);

  Serial.printf("Left: %d  | Right: %d\n", left, right);

  delay(200;

}
```

4. Uploading Code to Otto

1. Connect Otto via USB-C.
2. In Arduino IDE → Tools → Board → ESP32 Dev Module.
3. Tools → Port → select the detected COM port.
4. Upload Speed: 921600 (safe). Flash Mode: QIO.
5. Click Upload. If it fails at 'Connecting…', hold the BOOT button and release when uploading starts.

5. Third-Party Libraries & Links

NeoPixel (WS2812/WS2812B): Adafruit NeoPixel – https://github.com/adafruit/Adafruit_NeoPixel

OLED SSD1306 (I2C): Adafruit SSD1306 + Adafruit GFX – https://github.com/adafruit/Adafruit_SSD1306

MP3 Player: DFRobot DFPlayer Mini – https://github.com/DFRobot/DFRobotDFPlayerMini

OneWire: https://github.com/PaulStoffregen/OneWire

Servos: ESP32Servo – https://github.com/madhephaestus/ESP32Servo